MixBytes()

Gearbox Kelp Integration Security Audit Report

JUNE 17, 2026

Table of Contents

1. Introduction

1.1 Disclaimer

The audit makes no statements or warranties regarding the utility, safety, or security of the code, the suitability of the business model, investment advice, endorsement of the platform or its products, the regulatory regime for the business model, or any other claims about the fitness of the contracts for a particular purpose or their bug-free status.

1.2 Executive Summary

Gearbox Protocol's Kelp integration enables Credit Account users to interact with Kelp's rsETH. Users deposit supported assets into Kelp's LRT Deposit Pool to mint rsETH tokens. When redeeming rsETH for underlying assets, users initiate withdrawal requests that are queued in Kelp's withdrawal manager and subject to a delay period before becoming claimable. The integration employs a gateway architecture where KelpLRTWithdrawalManagerGateway creates per-account helper contracts that interact with Kelp's withdrawal queue and handle the claiming of matured withdrawals. Throughout the withdrawal lifecycle, pending and claimable positions are represented via KelpLRTWithdrawalPhantomToken, allowing Gearbox to value these delayed withdrawals as collateral by reflecting both locked and unlocked request states in terms of the output asset.

This audit was performed over 3 days by 3 auditors and included comprehensive manual code review and automated static analysis.

During the audit, in addition to reviewing common attack vectors and our internal security checklist, we conducted a thorough analysis of the following areas:

- **Withdrawal Contract Residual Balance Accessibility.** After all withdrawal requests have been processed, any tokens that remain on the KelpLRTWithdrawer contract can still be recovered. We confirmed that the gateway allows the rightful user to withdraw these residual tokens once every request has been completed.
- **Phantom Token Collateral Accounting.** The KelpLRTWithdrawalPhantomToken includes assets already held by KelpLRTWithdrawer in its balanceOf() calculation. We validated that these amounts are correctly counted in the Credit Account's collateral, preventing users from artificially lowering their health factor.
- **Handling of Native ETH and WETH Conversions.** The integration relies on seamless wrapping and unwrapping between ETH and WETH. We confirmed that KelpLRTWithdrawer and KelpLRTDepositPoolGateway correctly manage these conversions—wrapping incoming ETH to WETH, unwrapping when required, and accounting for balances accurately—so that no user funds are lost or misattributed.
- **Correctness of Kelp Protocol Interactions.** The integration makes external calls to Kelp's deposit and withdrawal contracts (LRTDepositPool and LRTWithdrawalManager). We confirmed that all interactions, including deposit operations, withdrawal initiation, and withdrawal completion, are correctly implemented with proper parameter passing and interface compliance, ensuring reliable communication with the Kelp protocol.

- **Isolation of Per-Account Withdrawal Contracts.** The KelpLRTWithdrawer contracts are per-account helper contracts created via minimal proxy clones, with each Credit Account having its own dedicated withdrawer instance. We verified that all functions on KelpLRTWithdrawer are protected by the gatewayOnly modifier, ensuring that only the KelpLRTWithdrawalManagerGateway can interact with these contracts on behalf of Credit Accounts, and that unauthorized users cannot directly execute withdrawal operations or access funds belonging to other accounts.
- **Access Control for Administrative Functions.** The integration includes configuration functions that manage allowed assets and token mappings, such as setAssetStatusBatch() in KelpLRTDepositPoolAdapter and setTokensOutBatchStatus() in KelpLRTWithdrawalManagerAdapter. We confirmed that all administrative and configuration functions are protected by the configuratorOnly modifier, ensuring that only authorized configurator addresses can modify critical integration parameters and that these functions cannot be called by unauthorized users.

Integrations with delayed withdrawal protocols, such as Kelp, present a known challenge with pending withdrawal requests. The Gearbox team is aware of this limitation and has implemented specific mitigation mechanisms. When a Credit Account's collateral is largely represented by a non-transferable phantom token whose underlying is still pending/locked, that collateral cannot be immediately realized on-chain. As a result, liquidation flows that rely on unwrapping the phantom position may revert and be delayed until withdrawals become claimable. To address this, Gearbox uses safe prices for such positions and enforces a minimum health factor threshold that prevents accounts close to liquidation from initiating delayed withdrawals to avoid liquidation during volatile periods.

The underlying Kelp Protocol contracts (LRTDepositPool, LRTWithdrawalManager, and related components) were not audited as part of this audit. However, all interactions between Gearbox and Kelp contracts were thoroughly analyzed, including interface verification, integration patterns, and the security implications of external calls to Kelp's withdrawal queue and deposit mechanisms.

Overall, this integration demonstrates a solid implementation that adheres to security best practices. The architecture maintains clear separation between adapters, gateways, and phantom token components, with each layer functioning effectively within the integration. The codebase maintains high quality standards and security posture, and no severe vulnerabilities or critical issues were discovered during the review.

1.3 Project Overview

Summary

Title	Description
Client	Gearbox
Category	Lending
Project	Kelp Integration

Title	Description
Type	Solidity
Platform	EVM
Timeline	19.12.2025 - 17.06.2026

Scope of Audit

File	Link
contracts/adapters/kelp/KelpLRTDepositPoolAdapter.sol	contracts/adapters/kelp/KelpLRTDepositPoolAdapter.sol
contracts/adapters/kelp/KelpLRTWithdrawalManagerAdapter.sol	contracts/adapters/kelp/KelpLRTWithdrawalManagerAdapter.sol
contracts/helpers/kelp/KelpLRTDepositPoolGateway.sol	contracts/helpers/kelp/KelpLRTDepositPoolGateway.sol
contracts/helpers/kelp/KelpLRTWithdrawalManagerGateway.sol	contracts/helpers/kelp/KelpLRTWithdrawalManagerGateway.sol
contracts/helpers/kelp/KelpLRTWithdrawalPhantomToken.sol	contracts/helpers/kelp/KelpLRTWithdrawalPhantomToken.sol
contracts/helpers/kelp/KelpLRTWithdrawer.sol	contracts/helpers/kelp/KelpLRTWithdrawer.sol

Versions Log

Date	Commit Hash	Note
19.12.2025	9d3c72a06d82c52418fa86b8d4a8385052af7727	Initial Commit
09.01.2026	646ed933878a0acced22b675d5f1e02e6c33655b	Commit for the reaudit
12.01.2026	047163d347febcfdd09a609edfe192355a6ba529	Commit with updates
17.06.2026	48efe2f73c00bcb34976ae29768c32a8321c6e8b	Immunefi fix commit

Mainnet Deployments

Deployment verification will be conducted via
https://permissionless.gearbox.foundation/bytecode/

1.4 Security Audit Methodology

Stage 1: Interim Audit

Project Architecture Review

OBJECTIVE: UNDERSTAND THE OVERALL STRUCTURE OF THE PROTOCOL AND IDENTIFY POTENTIAL SECURITY RISKS, INCLUDING PRIMITIVES AND ABSTRACTIONS IMPLEMENTED BY THE SYSTEM, HOW THEY INTERACT ARCHITECTURALLY, AND WHERE DESIGN OR INTEGRATION WEAKNESSES COULD BE EXPLOITED.

Activities include:

- understanding overall system design and contract interactions
- mapping responsibilities of each component and protocol workflows
- conducting a thorough line-by-line review of contracts and modules
- tracing execution paths and mapping state transitions
- identifying trust boundaries and external integrations
- forming an independent architectural understanding directly from code before validating documentation
- running the project test suite to observe implementation behavior and encoded invariants
- reviewing documentation, specifications, READMEs, and deployment guides and reconciling them with code

Adversarial Code Review

OBJECTIVE: IDENTIFY POTENTIAL VULNERABILITIES SPECIFIC TO THE PROTOCOL ARCHITECTURE, BUSINESS LOGIC, AND ECONOMIC MECHANISMS.

Approach includes:

- analyzing the codebase from an attacker's perspective, focusing on what can fail rather than only confirming intended behavior
- searching for overlooked edge cases, invalid assumptions, unexpected call sequences, and unsafe state transitions
- attempting to violate protocol invariants and identifying scenarios where system guarantees break
- analyzing economic attack surfaces including liquidation logic, oracle dependencies, vault accounting, and cross-contract interactions
- writing targeted tests, modelling adversarial scenarios, applying mutation testing and fuzzing, and developing proof-of-concept exploits
- conducting independent exploit path discovery by each auditor to reduce anchoring bias
- reviewing high-risk code collaboratively in pair-auditing sessions led by the Team Lead

Systematic Vulnerability Analysis

OBJECTIVE: APPLY STRUCTURED ANALYSIS AND KNOWN ATTACK PATTERNS TO ENSURE COMPREHENSIVE COVERAGE ACROSS THE CODEBASE.

Activities include:
- reviewing code against an internally maintained vulnerability checklist derived from past exploits and research
- analyzing common DeFi attack classes such as reentrancy, accounting manipulation, oracle manipulation, privilege escalation, and state desynchronization
- using proprietary AI tooling to surface candidate issues across broader attack vectors
- manually validating and triaging AI-generated candidates

Consolidation of Auditors' Reports

OBJECTIVE: MERGE INTERIM INPUTS FROM ALL AUDITORS INTO A COHERENT REPORT WITH CONSISTENT SEVERITY LEVELS AND REPRODUCIBLE FINDINGS.

Activities include:
- cross-checking findings across auditors
- consolidating and deduplicating issues
- resolving discrepancies in severity assessments
- using proprietary AI tooling to assist with report drafting and consistency checks
- manually reviewing and finalizing the report narrative
- preparing the interim audit report for client review

Stage 2: Re-audit & Mainnet Deployment Verification

Re-audit

OBJECTIVE: VERIFY THAT CLIENT REMEDIATIONS ARE CORRECTLY IMPLEMENTED AND THAT THE UPDATED CODE DOES NOT INTRODUCE NEW VULNERABILITIES.

Activities include:
- confirming each remediation matches the original recommendation
- documenting rationale for any unresolved findings
- verifying modified logic and integration points remain secure
- executing the project's tests after remediation, including targeted coverage of fixed areas
- using proprietary AI tooling on the updated codebase to surface potential new issues

Mainnet Deployment Verification

OBJECTIVE: PERFORM FINAL VERIFICATION OF DEPLOYED CONTRACTS AND CONFIGURATION BEFORE ISSUING THE PUBLIC AUDIT REPORT.

Activities include:

- verifying deployed contract bytecode against audited source code across networks
- confirming deployed bytecode matches the build produced from the audited commit using identical compiler settings
- reviewing constructor and initializer arguments used in deployment
- verifying proxy deployment order, initialization flow, and admin configuration
- ensuring implementations are not left uninitialized or exposed to initialization front-running
- publishing the final report after alignment between client and audit team

1.5 Risk Classification

Severity Level Matrix

Severity	Impact: High	Impact: Medium	Impact: Low
Likelihood: High	Critical	High	Medium
Likelihood: Medium	High	Medium	Low
Likelihood: Low	Medium	Low	Low

Impact

- **High** – Theft exceeding 0.5% of the protocol's TVL, partial or complete blocking of funds on the contract without the possibility of withdrawal (>0.5%), or loss of user funds exceeding 1% for users interacting with the protocol.
- **Medium** – Contract lock that can only be resolved through a contract upgrade, one-time theft of rewards or an amount up to 0.5% of the protocol's TVL, or funds lock where withdrawal is still possible by an administrator.
- **Low** – One-time contract lock that can be resolved by an administrator without requiring a contract upgrade.

Likelihood

- **High** – An event with an estimated 50–60% probability of occurring within a year, which can be triggered by any actor (e.g., due to a market condition that the actor cannot influence).
- **Medium** – An unlikely event (10–20% probability of occurring) that can be triggered by a trusted actor.
- **Low** – A highly unlikely event that can only be triggered by the contract owner.

Action Required

- **Critical** – Must be fixed as soon as possible.
- **High** – Strongly advised to be fixed in order to minimize potential risks.
- **Medium** – Recommended to be fixed to improve security and stability.
- **Low** – Recommended to be fixed to improve overall robustness and efficiency.

Finding Status

- **Fixed** – The recommended fixes have been implemented in the project code and the issue no longer impacts the security of the protocol.
- **Partially Fixed** – The recommended fixes have been partially implemented, reducing the impact of the finding, but the issue has not been fully resolved.
- **Acknowledged** – The recommended fixes have not been implemented, and the finding remains unresolved or has been accepted by the project team without code changes.

1.6 Summary of Findings

Findings Count

Severity	Count
Critical	0
High	0
Medium	0
Low	2

Findings Statuses

ID	Finding	Severity	Status
L-1	Redundant Computation in KelpLRTWithdrawalPhantomToken.balanceOf()	Low	Fixed
L-2	Withdrawal Completion Blocked When Token Is Removed From Whitelist	Low	Acknowledged

2. Findings Report

2.1 Critical

Not Found

2.2 High

Not Found

2.3 Medium

Not Found

2.4 Low

L-1	Redundant Computation in KelpLRTWithdrawalPhantomToken.balanceOf()		
Severity	Low	**Status**	Fixed in 646ed933

Description

The balanceOf() function calculates the user's phantom token balance by calling getPendingAssetAmount() and getClaimableAssetAmount() on the KelpLRTWithdrawalManagerGateway. Both of these gateway functions delegate the computation to the same per-account withdrawer contract.

Inside the withdrawer, each call performs a traversal over all the user's withdrawal requests to determine whether requests are still pending or already claimable. As a result, for a single balanceOf() call, the same set of withdrawal requests is iterated twice, once to compute pending assets and once to compute claimable assets.

This duplication does not affect correctness or safety, but it introduces unnecessary external calls and repeated loops, increasing gas usage and reducing efficiency for a commonly accessed view function.

Recommendation

We recommend refactoring the logic to compute pending and claimable amounts in a single traversal of the withdrawal requests and reuse the results.

Client's Commentary:

Fixed in 646ed933878a0acced22b675d5f1e02e6c33655b

L-2	Withdrawal Completion Blocked When Token Is Removed From Whitelist		
Severity	Low	**Status**	Acknowledged

Description

In `KelpLRTWithdrawalManagerAdapter.completeWithdrawal()`, a check enforces that `asset` is present in `_allowedTokensOut`. If the configurator removes a token from this whitelist while there are pending withdrawals for that token, users cannot complete the withdrawal—they remain stuck until the token is re-added to the whitelist.

Recommendation

We recommend not restricting `completeWithdrawal()` and `withdrawPhantomToken()` by the `_allowedTokensOut` whitelist, allowing completion of already-initiated withdrawals even if the asset was later disallowed, ensuring that queued requests can always be finalized.

Client's Commentary:

Removing the whitelist check would allow control flow capture through the use of a custom token. Note that Kelp's contracts do not revert when a non-existing output token is passed to `userAssociatedNonces` or `nextLockedNonce`. This means that an attacker can send a custom token to the withdrawer, and then call `completeWithdrawal`, which will call `transfer` on that token (as there is existing balance on the withdrawer, even though the token is not supported by Kelp). As we want to avoid control capture as much as possible, it's simpler to keep the check intact. If any withdrawals become stuck due to actions by a curator, the curator can resolve them ad-hoc with users.

3. About MixBytes

MixBytes is a blockchain security firm specializing in the analysis of decentralized protocols and smart contract systems.

The company helps Web3 teams build resilient protocol architecture, smart contract logic, and economic mechanisms through a combination of AI-assisted analysis and senior human expertise.

Rather than focusing solely on one-time audits before deployment, MixBytes works with protocols across their entire lifecycle — from early architecture design to production audits and ongoing protocol evolution.

Our team consists of experienced security researchers, engineers, and protocol analysts with deep expertise in DeFi systems, adversarial protocol analysis, and smart contract security.

Over the years, MixBytes has worked with many of the most widely used protocols in the ecosystem, including **Lido**, **Aave**, **Curve**, **1inch**, **OKX**, **Mantle**, **Fluid**, **Gearbox**, **Resolv**, and others, helping teams identify vulnerabilities, strengthen protocol design, and improve the robustness of decentralized systems.

To enhance analysis efficiency and coverage, MixBytes also develops and uses internal AI-assisted tooling that helps surface potential risk signals during development and code review. These tools augment the work of senior auditors but do not replace expert analysis.

Protocol Security Lifecycle

MixBytes supports protocols across the full lifecycle of their development and operation.

- **Design Review.** Independent assessment of protocol architecture and economic design before critical decisions are embedded in production code.
- **AI Tooling.** AI-assisted analysis integrated into development workflows to surface potential risk signals during protocol development.
- **Smart Contract Audit.** Comprehensive manual verification of smart contract logic and protocol invariants before production deployment.
- **Security Retainer.** Continuous expert support for protocol upgrades, integrations, governance changes, and evolving attack surfaces after launch.

Contacts

https://mixbytes.io/

https://github.com/mixbytes/audits_public

https://x.com/MixBytes

hello@mixbytes.io